PY103

Encapsulation: owning object state

Today

Welcome to your next lesson: Encapsulation: owning object state.

I'm getting your personalized lesson ready—based on what you've learned, what's ahead, and the best way for you to keep growing.

Step by step, you're moving forward.

Imagine you have a Pet object that has a happiness level. You don't want the rest of your program randomly changing that happiness in messy ways. Instead, you want the pet itself to be in charge of its own data. 🐶🧠

That idea—"the object owns and manages its own data"—is the heart of encapsulation.

Quick check before we go deeper:

Does this match how you think about it so far?

Yes, I get the idea of "object owns its data"
Sort of, but I'm not fully sure yet
Not really, I'm confused

Just tell me which of the three fits you best, and we'll go step by step from there.

Yes, I get it

Nice 👌 that's a strong starting point.

Let’s make this more concrete with a tiny picture of how encapsulation will look in code for our Pet:

The pet object keeps its data inside (like _name, _happiness)
The main program doesn’t touch those directly
Instead, the main program asks the pet for info through a method (a getter), like get_happiness()

So the flow is:

main code → calls getter → getter reads internal attribute → value comes back

Before we touch any code:

Does that flow make sense, step by step, or should I draw this as a little visual next? 🧠👉📊

That makes sense

Great 👍

Let’s lock in the “object owns its data” idea with one clear piece: internal attributes.

In Python, we signal “this should be managed inside the object” by starting the attribute name with a single underscore, like:

```
_name
_happiness
```

So inside a Pet object, we might store state like this:

_name → the pet’s name
_happiness → a number for how happy it is

From the outside (the main program), we’ll treat these as “hands off” and use methods to read them instead.

Quick check:

If you see an attribute named _happiness on a pet object, what does that underscore suggest?

“This is meant to be managed internally by the object”
“Python will completely block all access to it”

Just tell me 1 or 2.

1 to be managed locally but i believe you expained earlier it not enforced its more a a gentelens code of honror , if me attribute is named _attribute use my methods only for access

Exactly 💯 — that’s a perfect way to say it.

Single underscore = “I’m internal. Please use my methods to access me.”

Not a hard wall, more like a gentle coder’s agreement.

Now let’s add the next piece on top of that: the getter.

Step by step, here’s what a getter does:

It’s a method on the class, like get_happiness(self).
Inside, it just reads the internal attribute (like self._happiness).
It returns that value to the caller.
It does not change the state — only reads.

So the main code will do:

Python
pet.get_happiness()
instead of:

Python

```
pet._happiness
Make sense so far: a getter = a method whose job is to read and return an internal attribute, without changing it?

If yes, I’ll show you a small Pet example next and have you try something in the editor.

Yes, show me the example

Nice, let’s keep it tight and clear 🔍


Here’s a small Pet example that uses:


internal attributes: _name, _happiness
getter methods: get_name(), get_happiness()
Python
class Pet:
    """A simple pet that has a name and a happiness level."""

    def __init__(self, name, happiness):
        """Initialize the pet with a name and a happiness value."""
        self._name = name          # internal attribute
        self._happiness = happiness  # internal attribute

    def get_name(self):
        """Return the pet's name."""
        return self._name

    def get_happiness(self):
        """Return the pet's happiness level."""
        return self._happiness


# --- main program starts here ---

my_pet = Pet("Buddy", 80)

print(my_pet.get_name())        # uses getter
print(my_pet.get_happiness())   # uses getter
Key idea: the main program never touches my_pet._happiness directly — it asks via get_happiness().

Your turn:

```

Open the code editor, paste this code, run it, and tell me:

What output do you see?
Then change the name "Buddy" to a different name and run again. What changed in the output?
Buddy
80

Nice work running that 🎯

So with "Buddy", 80 you saw:

first line: the pet’s name (from get_name())
second line: the pet’s happiness (from get_happiness())

Both values came from internal attributes: _name and _happiness.

Next tiny step:

In the editor, change the code so you create the pet like this:
Python
my_pet = Pet("Shadow", 95)
Run it again.
Tell me just the new output you see (both lines).
Shadow
95

Perfect 🌟

You’ve just seen encapsulation in action:

_name and _happiness are internal attributes (with the underscore).
The main program only talks to them through get_name() and get_happiness().

So this call:

Python

my_pet.get_happiness()
follows the pattern:

main code → getter method → internal attribute → value returned

Quick check on understanding (just answer in your own words):

When you call my_pet.get_happiness(), which one of these is true?

A) It reads the _happiness value and returns it.
B) It changes the _happiness value inside the object.
it reads the _happiness value and returns it

What do you think the right answer is? Pick one option in the question below 👇

What does the get_happiness() method do in our Pet class example?

A.
It reads _happiness and returns it.

B.
It changes _happiness to a new value.

C.
It deletes _happiness from the object.

D.
It creates a new pet object.

Yes, exactly right 🎯

get_happiness() reads the internal attribute _happiness and returns that value. It doesn't change, delete, or create anything — it's a getter, so its only job is to give you the current state.

So in this line:

Python
print(my_pet.get_happiness())
the flow is:

print → calls get_happiness() → get_happiness() reads self._happiness → returns it → print shows it.

Next, I'd like to zoom in on the encapsulation pattern itself:

reading through a getter vs. touching the attribute directly.

Ready for a tiny compare-and-contrast example between:

my_pet._happiness
my_pet.get_happiness()

?

Yes, show me the comparison

Nice, let's keep this really clear and visual 🧠✨

Think of two different ways to get the happiness value:

Direct attribute access (not following encapsulation):
Python
print(my_pet._happiness)
Main code reaches inside and touches _happiness directly.
Ignores the "please use my methods" convention.

Getter access (follows encapsulation):
Python
print(my_pet.get_happiness())
Main code talks to the method.
The method talks to _happiness and returns the value.
Respects the idea that the object manages its own state.

You'll get the same number printed, but option 2 is the

encapsulation-friendly way.

Quick check:

If you want to follow the encapsulation pattern we're learning, which style should the main program prefer to use?

A) my_pet._happiness
B) my_pet.get_happiness()

Just answer A or B.

b

What do you think is the best choice here? 👇

If you want to follow the encapsulation pattern, which is the better way for the main program to read a pet's happiness?

A.
my_pet._happiness

B.
my_pet.get_happiness()

Exactly 💪

my_pet.get_happiness() is the encapsulation-friendly way because:

The main program talks to a method (the getter)
The object itself decides how to store and manage _happiness
From outside, you only see the clear, simple interface: "get the happiness"

Even though _happiness is still technically reachable, using the getter respects the "owner of the data is the object" idea.

Now let's add one more getter to your Pet to practice.

In the editor, do this step by step:

Start from this class (you can reuse your existing one):
Python

```
class Pet:
    """A simple pet that has a name and a happiness level."""

    def __init__(self, name, happiness):
        """Initialize the pet with a name and a happiness value."""
        self._name = name
        self._happiness = happiness

    def get_name(self):
        """Return the pet's name."""
        return self._name

    def get_happiness(self):
        """Return the pet's happiness level."""
        return self._happiness


# --- main program starts here ---

my_pet = Pet("Shadow", 95)

print(my_pet.get_name())
print(my_pet.get_happiness())
```

In the class, add a new getter called get_info(self) that:

returns a sentence like "Shadow is 95 happy" using self._name and self._happiness.
In the main program, add one more line:

Python

```
print(my_pet.get_info())
```

Run it, then tell me what the third line of output looks like.

```
Shadow
95
Shadow is 95 happy
```

Love it, that’s exactly what I hoped you’d see 🎉

Your output:

Text
Shadow
95
Shadow is 95 happy
matches this flow:

get_name() → reads _name → prints Shadow
get_happiness() → reads _happiness → prints 95
get_info() → reads both _name and _happiness and returns a nice combined message

All three methods are only reading internal state, not changing it — perfect encapsulation for this lesson. 🧰

I want to check one last key idea with you: the role of _happiness.

In our Pet:

_happiness is an internal attribute
get_happiness() is the getter that reads it

Does it feel clear now how those two work together, or do you want one more quick example before a final mini quiz?

That’s clear now

What do you think? Pick the best answer in the question below 👇

Your pet’s happiness is stored in _happiness. Which action best follows today’s encapsulation pattern?

A.
Read it through pet.get_happiness()

B.
Replace it with a subclass

C.
Put it in a list and loop over it

D.
Override __str__

Yes! 🎉 You picked the perfect answer:

Read it through pet.get_happiness()

That’s exactly the encapsulation pattern you’ve been using:

_happiness is an internal attribute (owned by the object)
get_happiness() is the getter that safely reads and returns that value to the outside world
The main program doesn’t poke _happiness directly — it asks the pet instead

You’ve now:

Grasped what encapsulation means (object owns its state)
Used internal attributes like _name and _happiness
Written and used getter methods to read that state ✅

You’re in a great spot to move on to the next lesson and build on this.

When you’re ready, use the Next Lesson button or go to Learn and continue your path. I’ll be right here when you want to dive deeper 🧠💻

```
class Pet:
    """A simple pet that has a name and a happiness level."""

    def __init__(self, name, happiness):
        """Initialize the pet with a name and a happiness value."""
        self._name = name          # internal attribute
```

```
            self._happiness = happiness  # internal attribute
    
        def get_name(self):
            """Return the pet's name."""
            return self._name
    
        def get_happiness(self):
            """Return the pet's happiness level."""
            return self._happiness
    
        def get_info(self):
            return f"{self._name} is {self._happiness} happy"
```